

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By

Cao Trần Khánh Linh - 23BI14252

Title:

**GEO-AWARE WEBASSEMBLY (WASM)
SERVERLESS ORCHESTRATOR**

External Supervisor: Eng. TRẦN TUẤN ANH

Internal Supervisor: Assoc. Prof. TRẦN GIANG SƠN

Hanoi, June 2026

To whom it may concern,

I, Trần Tuấn Anh, certify that the thesis/internship report of Ms. Cao Trần Khánh Linh is qualified to be presented in the Internship Jury 2025-2026.

Hanoi,

Supervisor's signature

Trần Tuấn Anh

ACKNOWLEDGEMENTS

First of all, I would like to express my sincere gratitude to the Department of Information and Communication Technology at the University of Science and Technology of Hanoi for providing the academic environment and technical foundation required to complete this thesis.

I would like to thank Associate Professor Tran Giang Son, my internal supervisor, for being a constant source of motivation. His interesting teaching method has attracted me to focus on every of his lectures. Besides, I am deeply thankful to my external supervisor and leader, Mr. Tran Tuan Anh, for guiding me from my very first day until I found my footing. Thanks to him and all my colleagues in FHN.NGT, my time at FPT Software has been so much more than just a normal internship. The knowledge I earned through real-world projects has been invaluable, and it contributed a massive part to the progress of this thesis.

I would also like to thank my partner for his endless encouragement. Throughout our three years studying together, he has been by my side for every group assignment, stood with me through the times I failed, cheered me up whenever I was burnt out, and shared all the fun we have had at USTH. This journey has been unforgettable, and more unforgettable with him.

To Derakkuma and Chuni Penguin: without you two, I might have actually studied harder, probably wouldn't have failed certain courses, and definitely would have saved a lot of money on retake fees.

To my classmates and friends, thank you for the shared struggles and your constant encouragement throughout the most difficult implementation stages of this project.

Most importantly, I want to express my deepest love and gratitude to my family. Thank you for your unconditional support, for believing in me even when I doubted myself, and for always giving me a safe place to come home to. You are the foundation of everything I do, and this accomplishment is as much yours as it is mine.

LIST OF ABBREVIATIONS

ABI	Application Binary Interface
ACR	Azure Container Registry
API	Application Programming Interface
CA	Certificate Authority
CI	Continuous Integration
CLI	Command-Line Interface
CPU	Central Processing Unit
DNS	Domain Name System
HA	High Availability
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JIT	Just-In-Time
JSON	JavaScript Object Notation
mTLS	Mutual Transport Layer Security
OCI	Open Container Initiative
RAM	Random Access Memory
SAN	Subject Alternative Name
SDK	Software Development Kit
SHA	Secure Hash Algorithm
TLS	Transport Layer Security
TTL	Time To Live
UI	User Interface
VM	Virtual Machine
WAL	Write-Ahead Logging
WASI	WebAssembly System Interface
WASM	WebAssembly

TABLE OF CONTENTS

SUPERVISOR CERTIFICATION	i
ACKNOWLEDGEMENTS	ii
LIST OF ABBREVIATIONS	iii
ABSTRACT	viii
I INTRODUCTION	1
1.1 Background	1
1.2 Limitations of Container-centric Orchestration	2
1.3 WebAssembly as an Edge Serverless Runtime	3
1.4 Research Problems	4
1.5 Contributions	4
II OBJECTIVES	6
2.1 General Objective	6
2.2 Functional Objectives	6
2.3 Non-Functional Objectives	7
2.4 Scope and Constraints	8
III MATERIALS AND METHODS	9
3.1 Technology Stack	9
3.2 Repository Organization	10
3.3 Master Control Plane Design	11
3.4 Worker Data Plane Design	13
3.5 WebAssembly Runtime and ABI Method	14
3.6 Supported Workload Model	15
3.7 Geo-Aware Scheduling Method	15
3.8 ACR and JIT Module Provisioning Method	16
3.9 Security	16
3.10 Experimental Setup and Evaluation Method	17
IV RESULTS AND DISCUSSION	20
4.1 Implemented Execution Lifecycle	20
4.2 Node Management and Registry Behavior	21
4.3 Concurrency Management in Go	22

4.4	Performance Architecture	23
4.5	Security	24
4.6	Operational Results	24
4.7	Limitations	26
V	CONCLUSION & PERSPECTIVE	27
	REFERENCES	28
	APPENDIX	30

LIST OF TABLES

Table 1:	Durable job state transitions in the master job store	12
Table 2:	Default worker execution and module-cache limits	14
Table 3:	Capacity-aware geo-routing: eligibility filters and scoring	16
Table 4:	Endpoint authentication and certificate identity requirements	17
Table 5:	Multi-region deployment topology on Microsoft Azure	18
Table 6:	Geo-aware routing: client coordinates map to the nearest worker	21
Table 7:	Worker-reported execution time (<code>execution_time_ms</code>) by module source and cache state	23
Table 8:	Master runtime metrics snapshot during evaluation	26
Table 9:	Master environment configuration	30
Table 10:	Worker environment configuration	31
Table 11:	wasmCat HTTP API surface	32

LIST OF FIGURES

Figure 1:	Use-case diagram	5
Figure 2:	Architecture diagram	11
Figure 3:	State machine diagram	13
Figure 4:	Module pushed to Azure Container Registry via ORAS	16
Figure 5:	Deployment diagram	18
Figure 6:	Creation and status retrieval of a durable execution job via wasmcctl .	20
Figure 7:	Sequence diagram	21
Figure 8:	Geo-routing: Requests with Southeast Asia coordinates route to the vn-01 worker, while US coordinates route to the west-us-01 worker.	21
Figure 9:	Operational workflow for safely draining an active worker node.	22
Figure 10:	Command-line output of the worker registry	22
Figure 11:	mTLS enforcement: A standard curl request is rejected	24
Figure 12:	Synchronous execution via CLI	25
Figure 13:	Browser dashboard	25
Figure 14:	Master node metrics output	25

ABSTRACT

Edge computing requires execution platforms that can place compute nodes as close to users as possible while preserving portability, isolation, and operational simplicity. Traditional container orchestration platforms, such as Kubernetes, provide mature scheduling and deployment mechanisms, but their dependency on container runtimes and complex control planes can be excessive for lightweight edge serverless workloads, and requires multiple setups in different geographical locations.

This thesis presents wasmCat, a Go-based WebAssembly serverless orchestrator designed for geo-aware execution across distributed worker nodes. wasmCat follows a master-worker architecture. The master accepts execution requests, tracks worker health, applies capacity-aware geo-routing, resolves module sources, and dispatches work over mutual TLS. Workers execute WebAssembly modules using the wazero runtime, enforce execution limits, manage digest-aware compiled module caching, and return invocation results through a defined memory ABI. The system also integrates Azure Container Registry module provisioning, SQLite-backed durable jobs, request idempotency, worker draining, metrics, a command-line interface, and a browser-based dashboard.

This thesis analyzes wasmCat as a serverless-style orchestration system for edge environments. It examines the theoretical motivation for WebAssembly over containers, the implementation of node management and scheduling in Go, the security boundaries created by mutual TLS, and the remaining limitations before production deployment.

Key words: WebAssembly, serverless, edge computing, orchestration, geo-routing, mTLS

I/ INTRODUCTION

1.1 Background

Modern distributed systems are being affected by the need to move computation closer to the location where data is produced and consumed; however, in conventional cloud architectures, user requests are typically routed to cloud regions where application servers and databases are concentrated. While this model is effective for large-scale resource pooling, it becomes less optimal when applications are sensitive to latency, bandwidth cost and data locality. Edge computing solves this limitation by deploying resources at geographically distributed locations, closer to end users, infrastructures, and regional networks [1].

The importance of edge computing is most visible in workloads that require real-time decision making [2], such as industrial monitoring, IoT gateways, local data filtering, and privacy-sensitive preprocessing. In such systems, the cost of sending all raw data to a distant cloud region may be higher than the cost of executing computations near the data source. For edge computing, the system must not only ask whether there is enough compute capacity, but also where that capacity is located relative to the user or device.

Serverless computing provides a complementary abstraction. In a serverless model, users submit functions or tasks rather than managing servers directly. The platform is responsible for receiving the invocation, selecting an environment, executing the function, and returning the result [3]. The term “serverless” does not mean servers do not exist; yet it means that server management is abstracted away from the developer. This abstraction is highly relevant to edge environments because developers should not need to manually connect to a specific edge node, install dependencies, start a daemon, or decide which node should execute each request.

wasmCat follows this serverless-style model. A user submits a WebAssembly module reference, a module name, user coordinates, and a payload, then the platform determines the worker node, fetches or reuses the module, executes it, and returns a structured result. The user does not need to choose a worker or operate the runtime directly. This makes wasmCat an orchestration layer rather than merely a WebAssembly runner.

As edge nodes are often geographically distributed and resource constrained, a practical platform must manage node registration, monitoring, scheduling, secure communication, module distribution, execution limits, and operational interfaces. wasmCat addresses these concerns through a Go-based master-worker architecture, in which the master maintains worker state and dispatches requests, while workers execute WebAssembly modules inside a sandboxed runtime. This separation features a control plane and data plane division that adapts to a lightweight serverless context.

1.2 Limitations of Container-centric Orchestration

Container orchestration platforms such as Kubernetes have become the standard approach for deploying distributed applications in cloud environments [4]. Containers provide packaging, filesystem and process isolation, network abstraction, and ecosystem compatibility. However, such features can create unnecessary complexity for small edge serverless workloads. A container-centric stack commonly assumes a container runtime, image registry, node agent, network overlay, service discovery mechanism, control plane components, and a declarative resource model. For full applications, these features are valuable, but for short-lived function-style execution at the edge, they may show excessive overhead.

The most direct limitation is startup cost, which can be understood as the latency overhead between a function request arrival and the beginning of code execution. In a containerized workload, it may require pulling images, unpacking filesystem layers, configuring namespaces, initializing networking, and managing daemon-level process state. Even when optimized, this lifecycle is heavier than instantiating a lightweight sandboxed module. And in edge environments, where functions may process small payloads and complete quickly, runtime overhead can dominate useful work. If a task takes only a few milliseconds to execute, the orchestration platform must avoid spending much more time preparing the environment.

A second limitation is installation complexity. If the orchestrator depends on Docker or containerd, then the edge node must support those systems. This may be acceptable in an enterprise-managed cloud cluster but will be less suitable for small nodes, or prototypes where the goal is to minimize dependencies. Meanwhile, wasmCat is distributed as native Go binaries: a master, a worker, a command-line control tool, and a web dashboard. The worker embeds its WebAssembly runtime through wazero, so there is no requirement for Docker, containerd, Kubernetes, or an external WebAssembly engine.

A third limitation concerns scheduling semantics. Traditional container orchestration often optimizes for cluster resource availability, affinity rules, and service health. Edge scheduling must also consider geographic proximity and potentially network latency. Round-robin placement or purely capacity-based scheduling can be insufficient when workers are distributed across regions. wasmCat therefore includes user latitude and longitude in the execution request and worker latitude and longitude in registration data. The scheduler first filters workers by state and resource availability, then selects the closest eligible worker using the Haversine formula [5]. This does not perfectly measure network latency, but it uses the assumption that shorter physical distance is associated with lower delay.

1.3 WebAssembly as an Edge Serverless Runtime

WebAssembly is a portable binary instruction format designed for safe and efficient execution [6]. Although it originated in the browser, WebAssembly has become increasingly relevant for server-side and edge computing because it provides a compact, sandboxed, language-neutral execution unit. A WebAssembly module can be compiled from languages such as C or Rust then executed by a compatible runtime on different host systems. This portability is valuable for edge computing, where worker nodes may vary in operating system, architecture, and installed dependencies.

The main advantage of WebAssembly for wasmCat is its execution model. A WebAssembly module executes inside a sandbox and interacts with the host through explicit imports, exports, and linear memory [7], does not directly access arbitrary host memory. This boundary is important for serverless systems because users submit executable modules that the platform must run safely. While WebAssembly alone is not a complete security solution, it provides a stronger default isolation model than loading native shared libraries into the host process.

WebAssembly also supports low startup overhead. In wasmCat, the worker stores compiled modules in a digest-aware cache to avoid recompiling them. The compiled module acts as a reusable execution template, while each invocation receives a fresh module instance with its own linear memory. This makes WebAssembly well suited to sub-millisecond warm-start execution, especially when compared with container startup paths that involve process and filesystem preparation.

wasmCat uses wazero [8], a WebAssembly runtime implemented in Go. Because wazero is embedded directly into the worker binary, the platform avoids foreign-function bindings to a C-based runtime and avoids requiring operators to install a separate execution engine. The runtime is created in `internal/worker/engine.go`, where the worker initializes a `wazero.Runtime`, module cache, in-flight compilation tracker, HTTP client, execution limits, and a semaphore for concurrency control.

The current wasmCat ABI is intentionally small. A module must export `memory`, `malloc`, and `run`. The host calls `malloc` to allocate space in the module's linear memory, writes the input payload using the wazero memory API, and invokes `run(ptr, len)`. The module returns a packed `uint64` containing the output pointer and output length. The host then reads that region back from linear memory. This ABI is simple enough for a final-year engineering system while exposing the essential mechanics of host-to-Wasm communication. Its limitation is strictness: modules that do not follow the expected exports or return invalid memory ranges fail at runtime.

1.4 Research Problems

The main concern of this thesis is how to design and implement a lightweight WebAssembly serverless orchestrator that can securely dispatch execution requests to geographically distributed edge workers without relying on a container runtime. This problem combines concerns from edge computing, serverless execution, distributed systems, runtime isolation, secure networking, and systems programming.

The first one is node management. The system must know which workers exist, where they are located, whether they are healthy and are available for new jobs. wasmCat resolves this through worker registration, periodic heartbeats, state tracking, and cleanup of stale workers. The master registry is implemented as an in-memory map protected by `sync.RWMutex`, while workers report CPU and RAM availability through heartbeat messages.

The second subproblem is scheduling. Given a request with user coordinates, the system must choose a suitable worker. wasmCat implements a capacity-aware geo-routing strategy: workers below CPU or RAM thresholds are filtered out, draining workers are excluded, and the closest remaining worker is selected using Haversine distance.

The third subproblem is secure module execution. The platform must fetch WebAssembly modules, verify digests when supplied, cache compiled modules safely, instantiate modules per request, and enforce execution limits. wasmCat addresses this in the worker engine using wazero, digest-aware cache keys, module size limits, payload limits, output limits, timeouts, and a concurrency semaphore.

Finally, operability should also be considered. A platform that requires long manual `curl` commands is difficult to use and demonstrate. Therefore, wasmCat includes a command-line and a user interface for browser-based operation. Both interfaces reuse the same master API and mTLS configuration model.

1.5 Contributions

This thesis has achieved several contributions. First, it presents a Go-native WebAssembly serverless orchestrator with a clear master-worker architecture. The master accepts execution requests, tracks workers, schedules placement, resolves module references, and dispatches work. The worker receives invocations, fetches and caches modules, executes WebAssembly, and reports results.

Secondly, the thesis implements geo-aware scheduling for edge execution. Instead of dispatching requests using a purely round-robin or random policy, wasmCat combines worker state, resource availability, and geographic distance. The use of the Haversine formula gives the scheduler an explicit spatial model suitable for edge computing, while leaving room for future network-latency-aware extensions.

Moreover, wasmCat integrates WebAssembly execution through wazero. The implementation demonstrates how Go can host a WebAssembly runtime, write payloads into linear memory, call an exported module function, read results, and manage compiled module caching. This provides a concrete serverless execution path without requiring containers.

Furthermore, the system includes security and reliability mechanisms. Internal communication uses mutual TLS. Worker identity is checked against certificate identity. Module digests can be validated. Durable execution jobs are stored in SQLite with request IDs, fingerprints, attempts, leases, and recovery behavior.

Finally, the project includes operator tooling. `wasmcatctl` provides a command-line interface for health checks, worker inspection, execution, jobs, metrics, and draining. `wasmcat-ui` provides a browser dashboard with equivalent operational capabilities through a backend proxy that preserves the mTLS boundary.

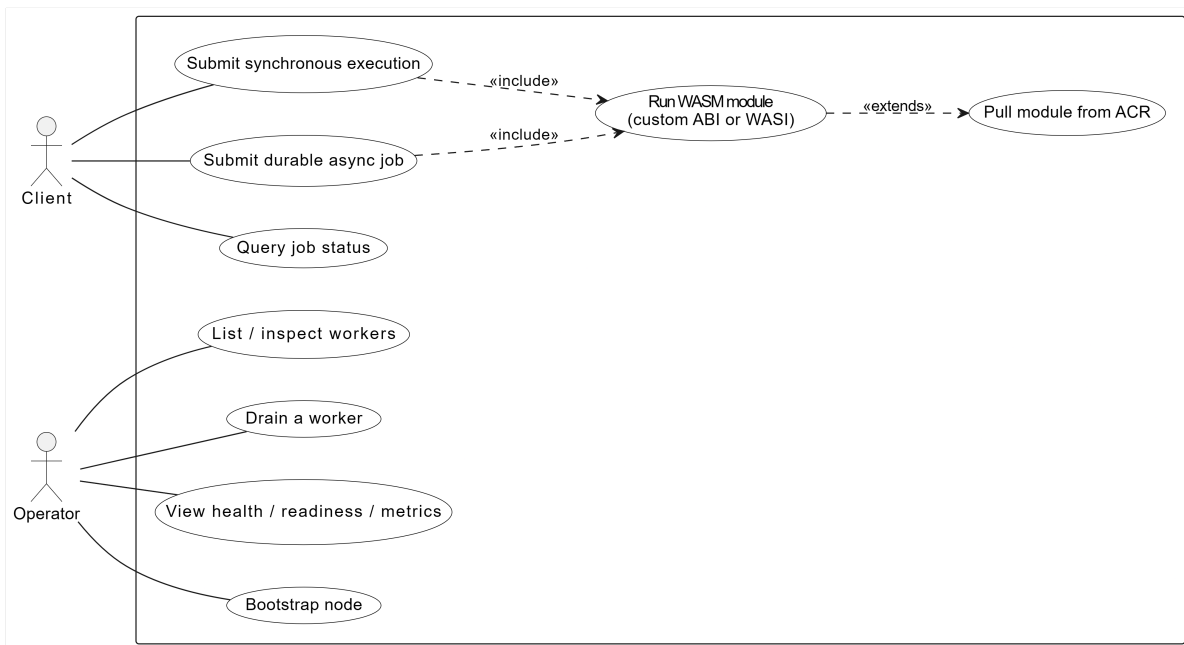


Figure 1: Use-case diagram

III/ OBJECTIVES

2.1 General Objective

The objective of this project is to design and implement a geo-aware WebAssembly serverless orchestrator for edge computing environments. The system aims to provide a lightweight execution platform in which users can submit WebAssembly codes and execute them without directly managing the worker nodes. The orchestrator will be responsible for receiving the request, scheduling an appropriate worker, provisioning the code, executing the workload inside a WebAssembly runtime, and returning the result.

This project is motivated by the limitations of container-centric orchestration for small, request-driven edge workloads. Rather than depending on Docker or Kubernetes, wasmCat dives deeper into whether a native Go-based orchestrator can provide the essential properties of serverless execution, including request-driven invocation, automatic placement, sandboxed execution, resource control, secure communication, and operational accessibility. The project combines practical systems engineering with reasonable concerns like runtime overhead, system design, and distributed trust boundaries.

The intended result is a prototype that demonstrates the core mechanisms required for WebAssembly-based edge serverless orchestration.

2.2 Functional Objectives

First of all, the primary goal is to build a complete system including a master node (control plane) and multiple worker nodes (data plane). The master will manage incoming execution requests from users and operator tools; as well as expose a synchronous execution API through a particular endpoint, validate the request structure, ensure a stable request identifier, apply module source policy, and forward the request to a suitable worker. Meanwhile, the worker will be capable of executing WebAssembly modules. Each worker must expose an invocation endpoint, receive execution requests from the master, fetch module bytes from a direct URL or registry-resolved blob URL (in this project, Azure Container Registry will be used), compile the module with wazero, instantiate it, transfer the payload into WebAssembly linear memory, call the exported "run" function, and return a structured response. The worker must also enforce execution limits so that invalid or excessive workloads will not consume unlimited resources.

Secondly, node registration and health tracking must be implemented. When online, workers must be able to register themselves with the master by providing their node identity, advertised address (a public IP or domain), and geographic coordinates. After registration,

workers must periodically send heartbeat messages containing CPU and RAM availability. The master will store these information in a registry, and with that, the master can verify their readiness, or use for scheduling or removing unhealthy workers when needed. This allows continuous node monitoring and ensures scheduling decisions are based on correct and latest information.

The system will also feature geo-aware scheduling. When a user request contains specific coordinates with latitude and longitude, the master node must select the most eligible worker based on both resources capacity and physical proximity. The scheduling method should filter out draining workers and workers that lack sufficient resources, then select the geographically nearest worker using Haversine distance formula.

Another functional objective is to support just-in-time module provisioning from Azure Container Registry. Rather than relying on local files, the master can detect Azure Container Registry module references, obtain repository-scoped pull tokens, fetch OCI manifests and select WebAssembly layers. After that, it can construct blob URLs, and forward digest information to workers.

Furthermore, to ensure reliability, the system supports durable execution jobs. The system must support asynchronous job creation, including request fingerprinting, status tracking, retry attempts, lease handling, and result persistence using SQLite. This allows users to look up job statuses and retrieve results more reliably.

Finally, operator interfaces are provided for better visibility and controls. The project will include a command-line interface called "wasmcatctl", and a browser dashboard, so that it will be easier for users to monitor the cluster health, metrics, workers, jobs, execution results, and worker draining behavior without having to write long curl commands in the terminal.

2.3 Non-Functional Objectives

The first non-functional objective is security. wasmCat must keep internal communication between master and workers secure using mutual TLS, so both sides can verify who they are talking to. The master should not accept a worker simply because it sends a valid HTTP request [9], but it also must verify client certificates and all worker actions, such as registration, health updates, requests must be tied to worker certificate identities. This prevents any attempt to join the cluster unauthorized or pretend to be another worker.

Another important goal is to keep the system easy to operate for users. The orchestrator should be installable as native binaries and not require a container runtime, such as containerd, Docker, etc. to operate. This directly supports the edge computing context, where nodes may be small and lightweight with limited CPU, memory, or storage.

The third non-functional objective is execution efficiency. Since compiling repeatedly would add unnecessary overhead, the worker caches compiled modules after they are first loaded. To prevent the cache to grow forever and reuse old codes, it should use digest-aware

cache keys to avoid module reuse and enforce cache lifecycle controls such as maximum entries, maximum cache size, and time-based eviction. This keeps warm executions fast while still reducing the risk of stale modules being executed.

The system also needs clear resource boundaries. A worker should not accept unlimited input, download oversized modules, produce unbounded output, or run too many executions at once. wasmCat therefore applies limits for payload size, module size, output size, execution duration, fetch duration, and concurrent execution count. This protects the worker process from accidental overload and forms a basic resource governance layer.

Finally, wasmCat must be observable to operate and debug. The master and workers expose health, readiness, and metrics endpoints, while logs are written in a structured format. Operator tools such as `wasmcatctl` and the web dashboard make cluster state easier to inspect during testing, demonstrations, and future production hardening.

2.4 Scope and Constraints

The scope of this thesis is limited to the initial development phase of the wasmCat orchestrator. The primary goal of this iteration was to build a functional baseline for WebAssembly edge execution, encompassing master-worker routing, ACR provisioning, mTLS security, and local job durability via SQLite.

As a developing system, several operational features have not yet been fully implemented. Specifically:

Distributed State: The control plane has not yet been developed into a highly available cluster. It currently operates as a single master node without leader election or distributed database consensus.

Identity and Tenancy: A robust user-management system has not been fully built. Features like per-tenant quotas, billing, and federated identity are planned for future iterations, with current security relying solely on mTLS certificates.

Automation and Ecosystem: The platform does not yet have the capability to autoscale worker nodes dynamically. Additionally, ecosystem features like module marketplaces, function versioning, and event triggers are currently undeveloped.

III/ MATERIALS AND METHODS

3.1 Technology Stack

wasmCat was implemented as a native Go system using Go 1.26.2 [10]. Go was selected because it provides high performance efficiency with low operational overhead with an accessible ecosystem for cloud-native infrastructure. The system is organized as a Go module with separate binaries for the control plane, data plane, command-line interface, and browser dashboard. The WebAssembly execution runtime used in the worker is wazero [8], a WebAssembly runtime implemented in Go. Since wazero is included as a Go library in the project, each worker binary contains the runtime needed to compile, initialize, and execute WebAssembly modules. Consequently, a worker node does not need to have any type of container runtime installed on the host.

For durable job persistence, the system uses SQLite [11] through the Go package `modernc.org/sqlite` [12], as it provides a lightweight, file-based persistence layer without requiring a separate database server. This allows the master to persist execution jobs, request fingerprints, attempts, errors, and final responses while preserving the simple installation.

The project also uses the Azure SDK for Go [13] to authenticate the master node when resolving modules stored in Azure Container Registry (ACR) [14]. Specifically, the `azidentity` package provides `DefaultAzureCredential`, which obtains an Azure Active Directory access token from the available environment. The master then exchanges this Azure AD token with ACR's OAuth endpoint to obtain a repository-scoped refresh token, and an ACR access token with pull permission for the target repository. This token is cached in memory by registry and repository, then forwarded to the selected worker as a short-lived Bearer token so the worker can download the resolved WASM blob directly from ACR.

Worker telemetry is collected using `gopsutil` [15], a Go library ported from Python's `psutil`, to obtain CPU and RAM availability. These metrics are periodically sent from each worker to the master through heartbeat requests, allowing the scheduler to avoid assigning new execution requests to overloaded workers. Moreover, the HTTP communication layer is implemented with Go's standard `net/http` package [16], which provides the HTTPS server and client primitives used by both the master gateway and worker API. Mutual TLS is implemented through Go's `crypto/tls` and `crypto/x509` packages [17], enabling certificate-based authentication between the master, workers, and operator clients without relying on an external service mesh or container networking layer.

3.2 Repository Organization

The implementation follows a modular organization that separates endpoints, orchestration logic, runtime execution, shared models, security, operator tooling, and tests. The master endpoint is located in `cmd/master`, while the worker endpoint is located in `cmd/worker`. Tools that can be used for operational purposes are also included: `cmd/wasmcatctl` provides a command-line interface, and `cmd/wasmcat-ui` provides a browser dashboard.

The `internal/master` package includes all the components for the control plane. It encapsulates the gateway for request ingress, the dispatcher for execution orchestration, and the scheduler for worker selection. To manage the cluster state and provide reliability, it also includes a worker registry and a durable job model backed by a SQLite job store. A background job recovery loop ensures that interrupted tasks are resumed, while the module source policy and ACR resolution logic handle secure WebAssembly module retrieval from private registries. To conclude, this package manages the complete lifecycle of a request from accepting user input, tracking real-time cluster health, to selecting the most appropriate workers, resolving module sources, dispatching tasks, and recording final outcomes.

The `internal/worker` package helps implement the data plane, features an embedded HTTP server to receive instructions and a WebAssembly engine powered by `wazero` for high-performance execution. The package enforces strict execution limits and utilizes a memory bridge for host-module data exchange. Additionally, it contains logic for metrics collection and a telemetry loop that periodically reports the worker's resource availability (CPU and RAM) back to the master. Its primary responsibilities involve receiving execution requests, fetching and compiling WebAssembly modules, managing a compiled module cache, executing code within an isolated environment, and reporting results or errors back to the control plane.

The `internal/shared` package defines common things that will be used across the project, including common HTTP models, response helpers, and standardized metrics structures. It also centralizes retry logic and establishes default server/client timeouts to ensure consistent network behavior. Moreover, the `internal/security` package manages the system's trust identity, providing certificate path helpers, utilities for development certificate generation, and logic for constructing mTLS-enabled clients and servers. The `internal/ctl` package implements a reusable operator client, which serves as the underlying communication layer for the `wasmcatctl` CLI tool, and similarly powers the web dashboard backend via `internal/ui`.

Finally, testing logic is included in a `tests/` folder to separate from production code, ensuring the platform is thoroughly validated through unit, integration, smoke, and interface-specific tests. Furthermore, "tests/integration" and "tests/smoke" folders provide end-to-end validation of system behavior.

3.3 Master Control Plane Design

The master is the control plane of wasmCat. Its responsibility is to maintain cluster state, receive execution requests, select and schedule workers, resolve module sources, and dispatch work securely. During initialization, the master loads configuration from environment variables, creates a worker registry, constructs a scheduler, opens a SQLite job store, constructs a dispatcher, and starts the gateway HTTP server.

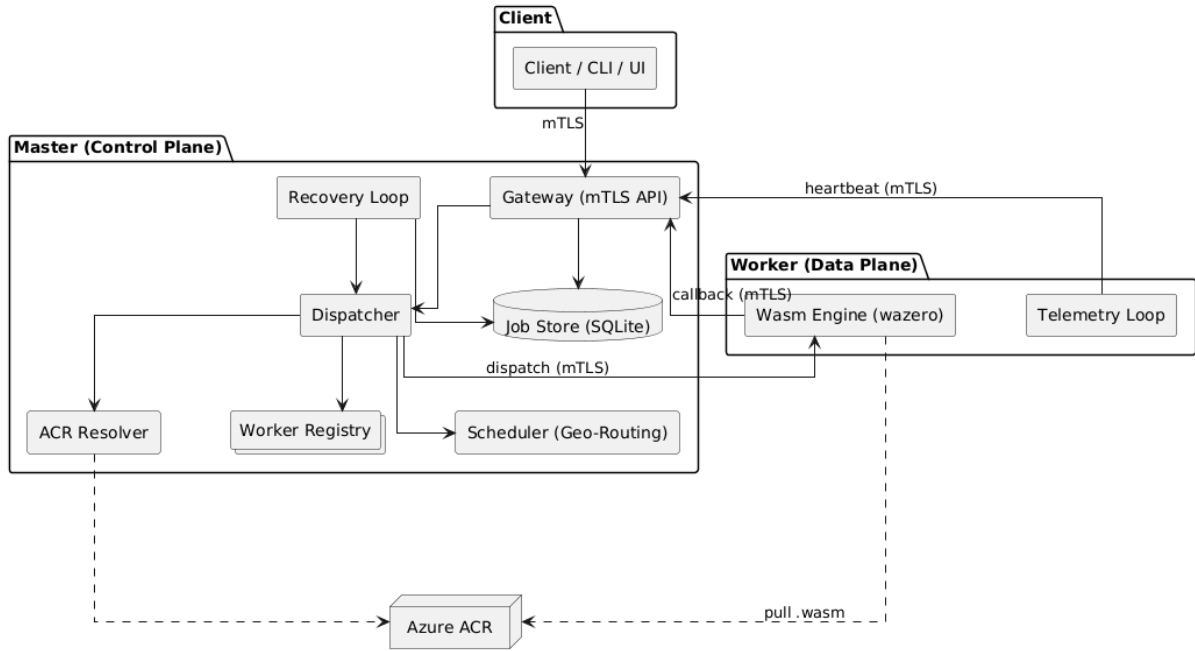


Figure 2: Architecture diagram

The master gateway exposes both public and internal APIs. Public operator-facing APIs include health, readiness, metrics, synchronous execution, durable job creation, durable job lookup, and worker listing. Internal APIs include worker registration, heartbeat reporting, worker draining, and job completion callbacks. The same gateway object coordinates request validation, method enforcement, body size limits, module policy checks, execution-client certificate validation, and job-store interactions.

Worker state is maintained in an in-memory registry. The registry stores worker ID, advertised address, geographic coordinates, CPU availability, RAM availability, last heartbeat time, and worker state. Access to this map is protected by `sync.RWMutex` [18], which implements CREW (Concurrent Read Exclusive Write), allowing concurrent readers at the same time while ensuring exclusive write during updates. This choice is made as scheduling will need to read worker state frequently, while registration and heartbeat updates mutate it periodically. Additionally, a background cleanup goroutine is implemented to remove stale workers whose last heartbeat exceeds the configured timeout.

The dispatcher coordinates execution placement, serving as the primary bridge between request ingress and data-plane execution. It begins by normalizing module URL fields and determining if the source requires resolution via the Azure Container Registry (ACR). When an ACR reference is detected, the dispatcher handles OCI manifest resolution and pulls necessary tokens using Azure Active Directory, forwarding a JIT bearer token to the worker. Once the module source is clarified, the dispatcher obtains candidate workers from the registry and invokes the scheduler to select the optimal node based on capacity and geographic proximity. After selection, the dispatcher forwards the request to the worker's `/invoke` endpoint over a secure mTLS connection, which ensures that only authenticated cluster components can trigger executions. To enhance reliability, the dispatcher implements conservative rescheduling logic. If a selected worker fails with a retryable HTTP status like `"502"` (Bad Gateway), `"503"` (Service Unavailable), or `"504"` (Gateway Timeout), the dispatcher removes that worker from the local candidate list and re-invokes the scheduler.

Durable jobs are implemented through a `JobStore` interface and a SQLite implementation. When the master receives a durable execution request, it computes a fingerprint from the request contents and creates a job record. The job record stores request ID, fingerprint, original request, status, worker ID, attempt count, maximum attempts, lease expiration, last error, optional response, and timestamps. The recovery loop periodically scans queued or expired jobs and attempts to recover them. This design provides a reproducible method for job reliability while avoiding the complexity of distributed consensus.

From	To	Trigger	Store operation
(initial)	queued	Client submits POST <code>/api/v1/jobs</code>	NewJobRecord
queued	dispatching	Recovery loop acquires a lease	MarkDispatching
dispatching	succeeded	Dispatch returns a result	MarkSucceeded
dispatching	queued	Retryable failure, attempt $<$ max	MarkQueued
dispatching	failed	Failure at attempt $\geq$ max	MarkFailed
dispatching	ambiguous	Lease expired before completion	MarkAmbiguous

Table 1: Durable job state transitions in the master job store

The `running` status is reserved for a future worker-completion protocol. The recovery loop treats a leased `running` job like `dispatching` for lease-expiry, but no transition currently enters it.

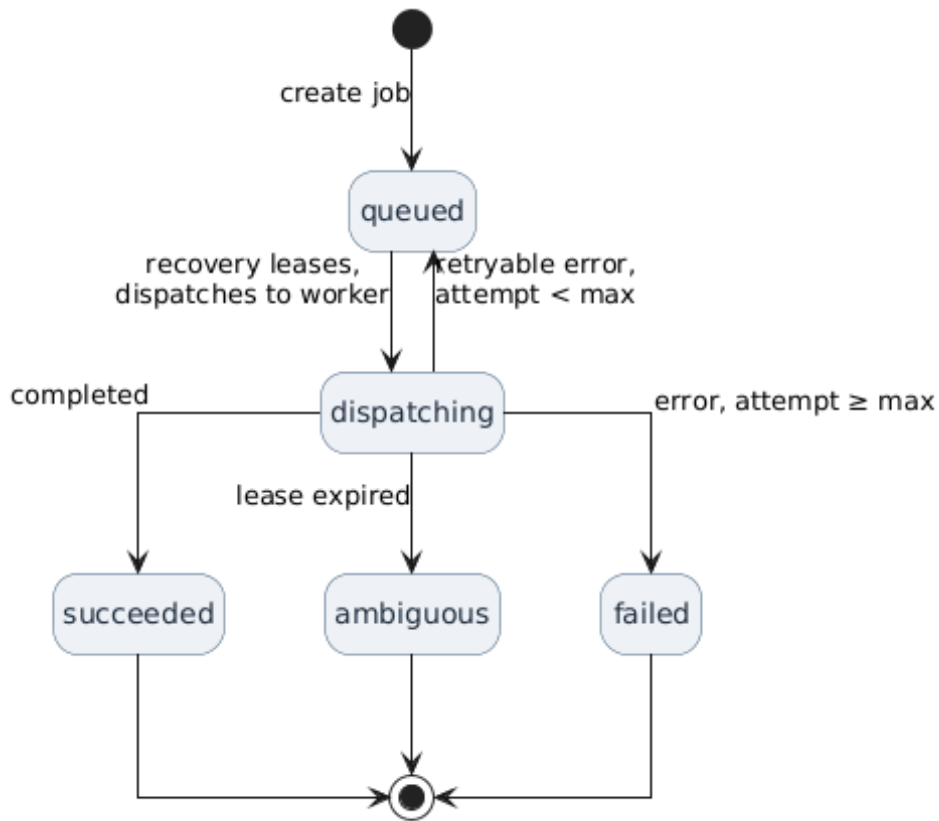


Figure 3: State machine diagram

3.4 Worker Data Plane Design

The worker process represents the data plane. Its primary responsibility is to execute WebAssembly modules on behalf of the master. During startup, the worker loads configuration, constructs a `WasmEngine`, creates a worker server, starts the telemetry loop, and starts an mTLS-protected HTTPS server.

The worker telemetry loop registers the node with the master and periodically sends heartbeat messages. Registration includes the worker ID, advertised address, latitude, and longitude. Heartbeats include information of CPU and RAM availability. This separation reduces repeated transmission of static node metadata while ensuring that scheduling decisions use current resource information. The master uses this telemetry to decide whether a worker is ready, stale, overloaded, or draining.

The worker server exposes health, readiness, metrics, and invocation endpoints. The `/invoke` endpoint receives execution requests forwarded by the master. Before execution, the handler enforces a request body limit, decodes the shared execution model, validates required fields, and ensures that the request contains a valid request ID. The handler then calls the WebAssembly engine using a bounded execution context.

The worker does not execute modules by spawning containers or external processes. Instead, it embeds the WebAssembly runtime directly in the Go process, which reduces operational complexity and removes the need for container runtime dependencies. The worker still enforces resource controls at the application level. These include maximum payload size, maximum module size, maximum output size, execution timeout, module fetch timeout, maximum concurrent executions, maximum cached modules, maximum cache bytes, and cache TTL.

Parameter	Default	Purpose
Execution timeout	5 s	Upper bound on a single run invocation
Module fetch timeout	10 s	Upper bound on remote .wasm download
Shutdown timeout	10 s	Graceful drain of in-flight work
Max module size	10 MiB	Rejects oversized module downloads
Max payload size	1 MiB	Bounds request input written to linear memory
Max output size	1 MiB	Bounds result read back from the module
Max concurrent execs	4	Semaphore limiting parallel executions
Max cached modules	128	Compiled-module cache entry cap
Max cache bytes	256 MiB	Compiled-module cache size cap
Module cache TTL	30 min	Eviction age for cached compiled modules

Table 2: Default worker execution and module-cache limits

Concurrency is controlled using a buffered channel as a semaphore. Each execution attempts to acquire a slot before proceeding. If all slots are occupied, the worker returns a capacity error. This mechanism is simpler than a full queueing system, but still able to prevent uncontrolled resource consumption and makes overload behavior explicit.

3.5 WebAssembly Runtime and ABI Method

The WebAssembly engine is implemented in `internal/worker/engine.go`. When a worker starts, it constructs a wazero runtime and initializes a compiled module cache which stores compiled modules. While compiled modules are reusable templates, each invocation receives a fresh instance with separate linear memory, therefore this approach reduces repeated compilation overhead while preserving per-request execution isolation.

After that, the execution process begins by validating payload size and acquiring the concurrency semaphore. It then creates an execution context with a timeout that is bound to the worker-side execution path, and if the module is not cached, the worker downloads the module bytes via HTTP. When a module digest is supplied, the worker verifies it before compilation. The supported digest format is "sha256:<hex>". The worker computes the SHA-256 digest of the downloaded bytes and compares it with the expected value. If a mismatch is found, compilation and caching process will be blocked, thus protect both integrity and cache correctness. If a registry tag changes and resolves to a different digest, the digest-aware cache key causes the worker to compile and cache a separate entry.

The ABI between the Go host and the WebAssembly module is also minimal. A valid module must export `memory`, `malloc`, and `run`. The host calls `malloc(size)` inside the module to obtain a pointer to a writable region of the module's linear memory. It then writes the request payload into that memory using wazero's memory API. After the input is written, the host calls `run(ptr, len)`. The module returns a packed `uint64`, where the high 32 bits represent the output pointer and the low 32 bits represent the output length. The host checks the output length against the configured maximum and reads the output bytes from linear memory.

In addition to this custom interface, the worker supports standard WASI modules [19], selected by an optional `abi` field on the request (`wasmcat`, `wasi`, or empty for auto-detection). A WASI (`wasi1`) module exports `_start`; the worker instantiates the `wasi_snapshot_preview1` host implementation provided by wazero, delivers the request payload on standard input, executes the module, and reads the result from standard output, treating an exit code of zero as success. When the `abi` field is empty, the worker inspects the compiled module's exports and

selects the custom ABI for a module that exports `run` or WASI for a module that exports `_start`. Both modes share the same digest-aware compiled-module cache and the same per-request instantiation model, so every invocation receives an isolated linear memory regardless of the selected mode.

3.6 Supported Workload Model

The execution model determines which programs can be deployed on wasmCat. The deployable unit is a single WebAssembly module that behaves as a stateless function: it receives one input payload, executes once to completion, and returns one output. In WASI mode, the worker provides the module with standard input and standard output, but it grants no network sockets, no filesystem, and no environment variables or process arguments. In custom-ABI mode, the module interacts with the host solely through linear memory. Consequently, any program that can be compiled to `wasm32-wasip1` (for instance from Rust, Go, TinyGo [20], C, or Zig) or to the custom ABI, and whose behaviour reduces to mapping an input to an output, can be deployed. Representative workloads include data parsing and transformation, encoding and decoding, compression, hashing and signature verification, input validation, and rule or expression evaluation.

This model also delimits what cannot be deployed. Because the sandbox exposes no sockets, a module cannot perform network communication; because no filesystem, environment, or arguments are supplied, programs that depend on them receive nothing; and because a module may import only `wasi_snapshot_preview1`, components that require further host interfaces, such as browser bindings or the WebAssembly component model and WASI-HTTP, are not supported by the current engine. Each invocation is further constrained by configurable limits on payload size, module size, output size, and execution time. wasmCat therefore targets stateless, request-scoped functions rather than long-running services or complete applications, which is the same unit of deployment adopted by edge function platforms and is a direct consequence of the lightweight, embedded-runtime design.

3.7 Geo-Aware Scheduling Method

The scheduling method combines capacity filtering with geographic distance. This reflects the central assumption of edge computing: the optimal worker is not merely the least loaded worker, but the least loaded worker that is also close to the user or data source. wasmCat represents this by requiring workers to register latitude and longitude, while execution requests include user latitude and longitude.

To start the process, the scheduler first receives the list of schedulable workers from the registry. Among them, workers that are in a draining state are excluded because they should not receive new executions. The scheduler then applies CPU and RAM thresholds.

A worker is considered eligible only if its reported free CPU percentage is greater than or equal to the configured minimum and its free RAM is greater than or equal to the configured minimum. This prevents the routing algorithm from selecting a physically close worker that is currently overloaded.

After capacity filtering, the scheduler computes geographic distance between the user and each eligible worker using the Haversine formula [5]. The Earth is approximated as a sphere with mean radius $R = 6371$ kilometers. Given user coordinates (lat_1, lon_1) and worker coordinates (lat_2, lon_2) , the values are converted from degrees to radians. The differences are computed as:

$$\Delta lat = lat_2 - lat_1$$

$$\Delta lon = lon_2 - lon_1$$

Stage	Rule	Description
1	State filter	Only workers in the ready state are eligible; draining workers are excluded
2	CPU filter	Workers below MIN_WORKER_CPU_FREE (default 0 %) are removed
3	RAM filter	Workers below MIN_WORKER_RAM_FREE_MB (default 0 MiB) are removed
4	Distance scoring	Remaining workers are scored by Haversine distance from user_lat/user_lon
5	Selection	The nearest eligible worker is selected

Table 3: Capacity-aware geo-routing: eligibility filters and scoring

The Haversine intermediate value is:

$$a = \sin^2\left(\frac{\Delta lat}{2}\right) + \cos(lat_1) \cos(lat_2) \sin^2\left(\frac{\Delta lon}{2}\right)$$

The angular distance is:

$$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$

The final distance is:

$$d = R \cdot c$$

The scheduler iterates through all eligible workers and selects the worker with the smallest distance.

3.8 ACR and JIT Module Provisioning Method

wasmCat supports just-in-time module provisioning from both direct module URLs and Azure Container Registry. When the dispatcher detects a module URL ending with `.azurecr.io`, it parses the registry name, repository name, reference, and reference kind. If the reference points to an OCI manifest [21], the master obtains a repository-scoped pull token using Azure Active Directory to exchange for an ACR access token scoped to `repository:<name>:pull`.

After authentication, the master fetches the OCI manifest and selects the WebAssembly layer. If multiple layers exist, the resolver searches for known WebAssembly media types such as `application/wasm`. The selected layer digest is then used to construct the blob URL. Then, the master forwards the blob URL, bearer token, and digest to the worker, and the workers only need to download bytes from a URL and compile the module.

```
azureuser@wasmcat-mCFLJ2I:~/demo$ oras repo tags wasmcatalog.azurecr.io/modules/jsonfmt
v1
```

Figure 4: Module pushed to Azure Container Registry via ORAS

3.9 Security

Security in wasmCat is based on mutual TLS [22]. In this case, both master and workers load a CA certificate and require verified client certificates. This means internal requests must be encrypted and authenticated

before reaching the handler logic. The master validates worker identity by comparing the certificate common name or DNS subject alternative names with the claimed worker ID [23]. Therefore, a worker certificate is not only a transport credential; it is bound to the worker identity which is used in registration, heartbeat, drain, and completion callback operations.

Endpoint	Port	Required client identity
/wasmcat/health ready metrics	7270	Any CA-trusted client certificate
/api/v1/execute, /api/v1/jobs	7270	CA-trusted cert; optional EXECUTE_CLIENT_ALLOWLIST
/api/v1/workers/{id}/drain	7270	Operator identity (allowlist-gated)
/internal/register heartbeat drain	7270	Worker cert bound to the requested worker ID
/internal/jobs/complete	7270	Worker cert bound to the worker ID
/invoke (worker)	7271	Master certificate

Table 4: Endpoint authentication and certificate identity requirements

The same security boundary applies to dispatch. The master forwards execution requests to workers using an mTLS HTTP client configured with the master certificate, master key, and CA certificate. Workers also use mTLS clients when registering, sending heartbeats, draining, and reporting job completion to the master. With this applied, it reduces the risk of unauthorized nodes joining the cluster or intercepting internal traffic.

Operator usability is implemented through two interfaces: `wasmcatctl` (a command-line tool) and `wasmcat-ui` (a browser dashboard). The browser dashboard preserves the mTLS boundary by avoiding direct browser-to-master communication. Instead, it calls a local UI backend, which loads the same configuration file and uses the same `internal/ctl` client to call the master over mTLS. These methods avoid exposing private client key while still providing a user-friendly operational interface. It also ensures that all actions follow the same API paths, validation rules, and security assumptions.

3.10 Experimental Setup and Evaluation Method

The system was deployed and evaluated on Microsoft Azure using three virtual machines distributed across two geographic regions, so that geo-aware routing could be tested. The master and one worker were placed in Southeast Asia, while a second worker was placed in West US 2; the two regional virtual networks were joined through global VNet peering, allowing the master to reach the remote worker over private addresses. Table 5 summarizes the node roles, regions, virtual machine sizes, and addresses of the deployed testbed. All nodes ran Ubuntu 24.04 and the native `wasmCat` binaries, and every node trusted a single certificate authority whose private key was held centrally in Azure Key Vault.

Role	Node ID	Region	VM size	Public IP	Private IP
Master	wasmcat-master-dev-01	Southeast Asia	Standard_D2s_v5	4.193.218.90	172.16.0.4
Worker	worker-vn-01	Southeast Asia	Standard_D2s_v3	–	172.16.0.5
Worker	worker-west-us-01	West US 2	Standard_D2as_v6	20.57.162.41	10.0.0.4

Table 5: Multi-region deployment topology on Microsoft Azure

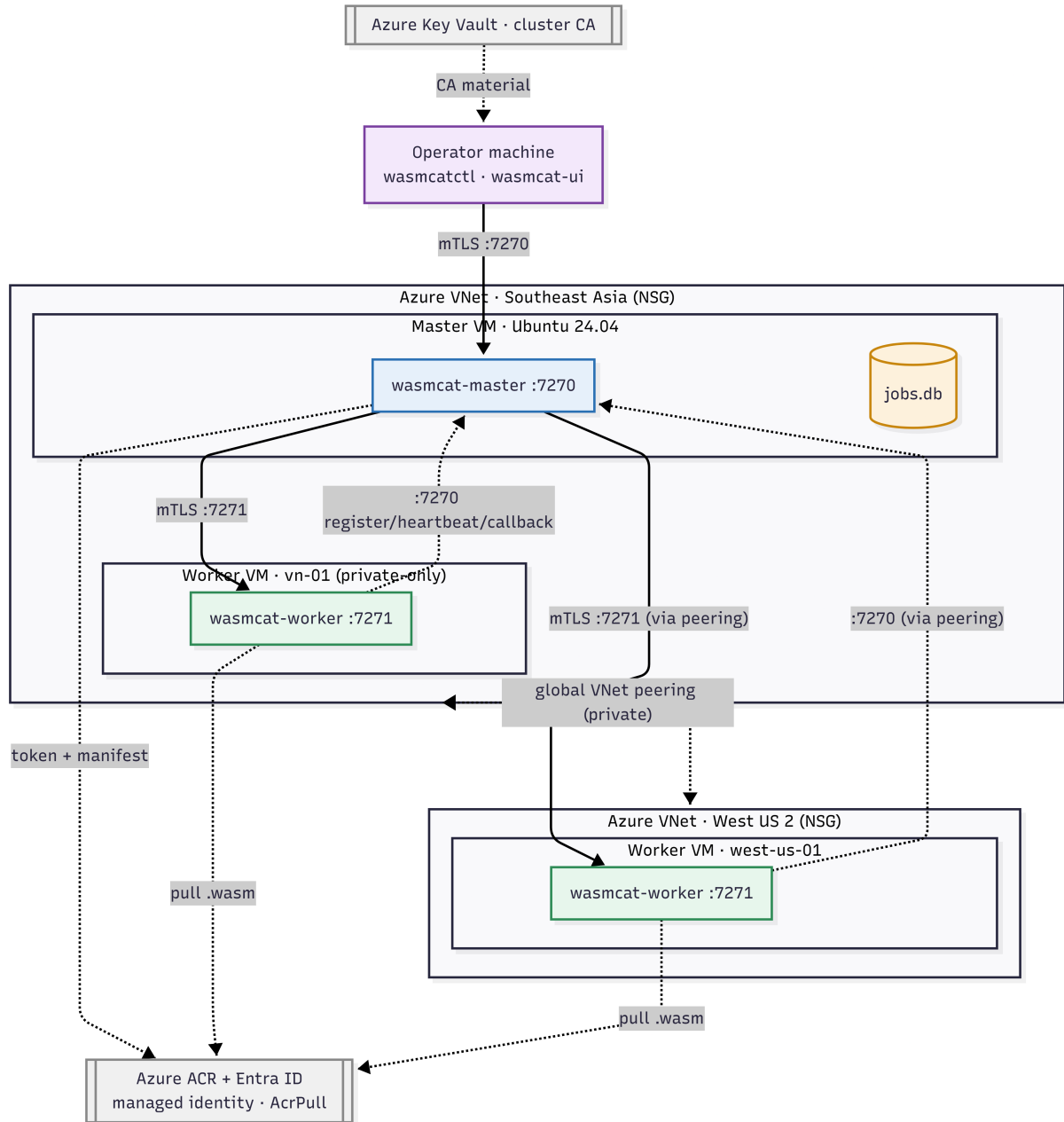


Figure 5: Deployment diagram

The workload used throughout the evaluation was a single WebAssembly module, `jsonfmt`, which parses a JSON payload and returns a canonically formatted JSON document. The module was compiled to WebAssembly and published to Azure Container Registry as an `application/wasm` OCI artifact using ORAS [24], so that the full just-in-time provisioning path was exercised end to end. The same module was also served from a direct HTTP

URL to allow the two provisioning sources to be compared.

All requests were issued with the `wasmcatctl` operator client, which reuses the same mutual-TLS configuration and API paths as any other client. Each execution response reports the worker that served the request (`executed_on_node_id`) and the worker-side execution time in milliseconds (`execution_time_ms`); the latter is the metric used for all latency figures reported in this work. Two cache conditions were distinguished: a *cold* measurement is the first execution of a module version on a worker, which includes remote download and wazero compilation, whereas a *warm* measurement is an immediately repeated execution of the same module, which is served from the compiled-module cache. Cold and warm measurements were collected for both the direct-URL and the ACR provisioning paths.

Geo-aware routing was evaluated by submitting identical requests with different coordinates and observing which worker the master selected. A request carrying coordinates near Singapore (1.29, 103.85) and one near Seattle (47.67, -122.12) were dispatched in turn, and the `executed_on_node_id` field of each response was recorded to confirm that the scheduler selected the geographically nearest eligible worker. Worker draining was verified separately by issuing a drain command through `wasmcatctl`, confirming the affected worker changed to the `draining` state in the registry, and observing that subsequent executions were routed to the remaining ready worker. Finally, mutual-TLS enforcement was checked by issuing a plain `curl` request without a client certificate and confirming that the master rejected it at the transport layer.

IV/ RESULTS AND DISCUSSION

4.1 Implemented Execution Lifecycle

Despite having certain constraints, the thesis has achieved an end-to-end execution lifecycle from user request submission to WebAssembly result delivery. The lifecycle begins when a user submits an execution request to the master through the command-line interface (CLI) or the web UI. wasmCat supports two execution paths: a synchronous execution path and a durable asynchronous job path.

In the synchronous path, the client sends a request to `POST /api/v1/execute` and waits while the master validates the request, selects a worker, dispatches the workload, receives the worker's result, and returns the final output in the same HTTP response. In the durable asynchronous path, the client sends the request to `POST /api/v1/jobs`. Instead of waiting for execution to complete, the master stores the request as a durable job in SQLite and returns a job record to the client. A background recovery loop later dispatches queued jobs. The client can query the job status through `GET /api/v1/jobs/{request_id}`.

```
azureuser@wasmcat-mcfl32i:~/demo$ wasmcatctl jobs create --module jsonfmt --registry-url https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1 --payload '{"j":1}' --request-id req_demo_1
request_id=req_demo_1
status=queued
worker=
attempt=0/3
last_error=
azureuser@wasmcat-mcfl32i:~/demo$ wasmcatctl jobs get req_demo_1
request_id=req_demo_1
status=succeeded
worker=
attempt=1/3
last_error=
result={
  "j": 1
}
```

Figure 6: Creation and status retrieval of a durable execution job via `wasmcatctl`

Both execution paths use the same request structure. After receiving the request, the master applies a module source policy to protect the execution pipeline. The system then uses the request ID and a request fingerprint to enforce idempotency across the distributed cluster. If the request ID is reused with the same execution content, the system returns the existing state. If reused for different content, the system treats it as a conflict.

The dispatcher then prepares the request for worker execution, resolves the ACR reference if needed, and uses the scheduler to select the geographically closest eligible worker. The master forwards the request to the selected worker over mutual TLS. The worker validates the request, fetches and compiles the module, instantiates the WebAssembly module, writes the payload into linear memory, calls the exported `run` function, reads the output, and returns a structured response.

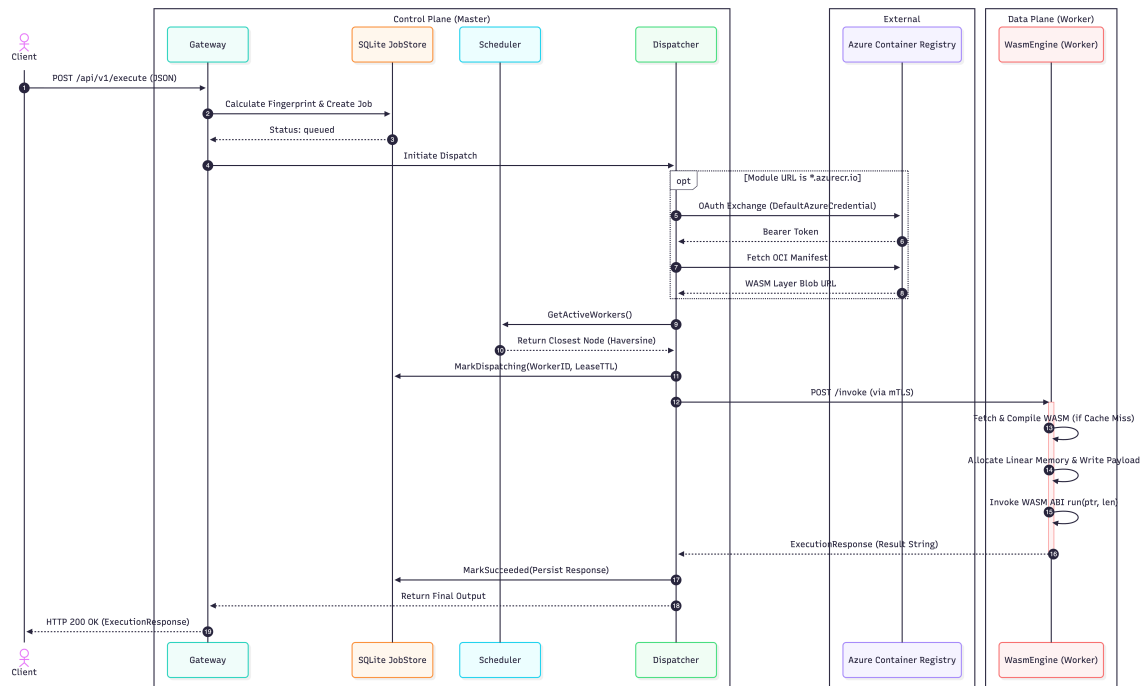


Figure 7: Sequence diagram

```

azureuser@wasmcat-MFL321:~/demo$ wasmcattcl -o json execute --module jsonfmt --registry-url "https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1" --payload '{"r":"sg"}' --user-lat 1.29 --user-lon 103.85
{
  "request_id": "req_b57a30c19fb0d016a91b93fa0bd84519",
  "result": "{\n  \"r\": \"sg\"\n}",
  "execution_time_ms": 2372.921,
  "executed_on_node_id": "worker-vn-01"
}
azureuser@wasmcat-MFL321:~/demo$ wasmcattcl -o json execute --module jsonfmt --registry-url "https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1" --payload '{"r":"us"}' --user-lat 47.67 --user-lon -122.12
{
  "request_id": "req_33594b9f9e85893d1893b9892e7669b9",
  "result": "{\n  \"r\": \"us\"\n}",
  "execution_time_ms": 3.392,
  "executed_on_node_id": "worker-west-us-01"
}
  
```

Figure 8: Geo-routing: Requests with Southeast Asia coordinates route to the vn-01 worker, while US coordinates route to the west-us-01 worker.

Client coordinates	Approx. location	Selected worker	Region
1.29, 103.85	Singapore	worker-vn-01	Southeast Asia
47.67, -122.12	Seattle	worker-west-us-01	West US 2

Table 6: Geo-aware routing: client coordinates map to the nearest worker

4.2 Node Management and Registry Behavior

Node management in wasmCat is built around the master registry. Workers register themselves with the master by providing a stable node identifier, an advertised network address, geographic coordinates, and initial resource information. After registration, each worker sends periodic heartbeat messages containing current CPU and RAM availability. This allows the master to maintain an up-to-date view of the cluster without holding a persistent connection to every worker.

The registry is implemented as an in-memory map protected by a read-write mutex using Go package “sync”. This means scheduling reads can proceed concurrently without blocking each other, while registration,

heartbeat updates, and cleanup operations acquire exclusive access. The access pattern is designed for the case that reads happen for every incoming execution request, while writes occur only on worker lifecycle events.

Workers can also exist in one of two operational states: ready and draining. A ready worker is eligible to receive new executions, and a draining worker remains visible in the registry but is excluded from scheduling. This gives operators a clean way to take a node out of rotation before shutting it down or performing maintenance, without dropping requests that are already in flight.

```
azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcctl drain worker-vn-01 && wasmcctl workers
status=success message=Worker marked as draining
ID          ADDRESS      STATE    CPU FREE  RAM FREE  LAST SEEN
worker-west-us-01  10.0.0.4:7271 ready    99.9%     7386 MB   0s ago
worker-vn-01      172.16.0.5:7271 draining 99.9%     7311 MB   0s ago
```

(a) Operator issuing the drain command to the vn-01 worker.

```
azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcctl workers
ID          ADDRESS      STATE    CPU FREE  RAM FREE  LAST SEEN
worker-west-us-01  10.0.0.4:7271 ready    99.9%     7334 MB   2s ago
worker-vn-01      172.16.0.5:7271 draining 99.4%     7288 MB   20s ago
azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcctl -o json execute --module jsonfmt --registry-url https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1 --payload '{"b":2,"a":1}'
{"request_id": "req_60e8972cfe407dbefcde8262effe963",
 "result": [{"a": 1, "b": 2}],
 "execution_time_ms": 2.932,
 "executed_on_node_id": "worker-west-us-01"}
```

(b) Registry output confirming the worker state has transitioned to 'draining'.

Figure 9: Operational workflow for safely draining an active worker node.

Moreover, the registry cleanup loop runs on a configurable interval and removes any worker whose last heartbeat exceeds the stale timeout. This prevents the scheduler from indefinitely selecting a node that has quietly disappeared. However, the main limitation of this design is that the registry is held entirely in memory, which means if the master process restarts, all worker membership is lost until workers re-register or send their next heartbeat. In real scenario, the five-second heartbeat interval means the cluster view is reconstructed quickly, but there is no durable membership record.

The GET /api/v1/workers endpoint makes the current registry state visible to operators. This allows the CLI to display active workers, their addresses, states, resource availability, and heartbeat recency without requiring direct database access.

```
azureuser@wasmcat-mCFLJ2I:~/wasmCat$ wasmcctl workers
ID          ADDRESS      STATE    CPU FREE  RAM FREE  LAST SEEN
worker-west-us-01  10.0.0.4:7271 ready    99.9%     7287 MB   4s ago
worker-vn-01      172.16.0.5:7271 ready    99.7%     7310 MB   2s ago
```

Figure 10: Command-line output of the worker registry

4.3 Concurrency Management in Go

Concurrency management is one of the more consequential implementation decisions in wasmCat because the system handles execution requests, worker heartbeats, cache operations, and background maintenance loops simultaneously. Rather than introducing an internal work queue, the design relies on Go's native HTTP server for per-request goroutines and uses explicit synchronization primitives for shared state.

On the master, incoming requests are handled concurrently by the standard library's HTTP server. Execution requests, heartbeat updates, metrics reads, and job queries may all arrive at the same time. The worker registry is protected by a read-write mutex so that many scheduling reads can happen in parallel while a heartbeat write or

a drain operation takes exclusive access. Two background goroutines run independently: one periodically scans for stale workers, and the other scans the durable job store for queued or expired jobs that need to be recovered.

On the worker, concurrency control is more tightly bounded because execution is a resource-intensive operation. WasmEngine uses a buffered channel as a semaphore, so every invocation must acquire a slot from this channel before proceeding. If all slots are occupied, the worker returns an immediate capacity error rather than queueing the request, keeping memory and CPU usage predictable.

The compiled module cache is also shared across concurrent invocations and protected by a mutex. To avoid a situation where multiple requests occur for the same cold module and compile identical ones, the engine uses an in-flight compilation map. When a module is not yet cached, the first request to arrive becomes the leader and registers an in-flight channel for that cache key. Subsequent requests for the same module wait on that channel instead of starting their own download, and once the leader finishes, the followers reuse the cached version, which prevents unnecessary cost.

4.4 Performance Architecture

This section reports representative cold and warm-cache latencies and analyzes where overhead is concentrated; large-scale load testing is left for future work. Workers in wasmCat do not start a new process or container for each request. There is no image pull, no filesystem layer unpacking, no namespace setup, and no cgroup configuration. A containerized cold start must pay at minimum the cost of unpacking filesystem layers, configuring process namespaces and cgroups, and initializing the application runtime. At the edge, where nodes are small and these operations are slower, that startup cost can easily dominate the useful work being done. wasmCat avoids this by embedding the wazero runtime directly in the worker binary. Each invocation operates inside the existing process.

The execution path separates into two cases depending on whether the module is already cached. On the cold path, the worker downloads the module bytes over HTTP, optionally verifies the SHA-256 digest, compiles the binary to native code through wazero, stores the compiled result in the cache, and then instantiates a fresh module to handle the request. The compile step is the most expensive part and happens only once per unique module version. On the warm path, the compiled module is retrieved from the cache and a fresh instance is created directly, skipping the download and compile stages entirely. Each instance receives its own linear memory, so warm cache reuse does not mean shared execution state between requests.

Module source	Cold (ms)	Warm (ms)
Direct HTTP URL	2135.559	7.529
Azure ACR (OCI manifest)	2372.821	2.932

Table 7: Worker-reported execution time (`execution_time_ms`) by module source and cache state

The cache key includes the module name, URL, and digest. This means a module that changes its content but keeps the same name or URL is still detected as a different version, because the digest will differ. The worker compiles and caches it separately rather than reusing a stale result.

The full request latency on the warm path still includes master-side validation and scheduling, the mTLS handshake and dispatch to the worker, module instantiation, the malloc and memory write for the payload, the actual function execution, output reading, response serialization, and the return network transit. Claiming sub-millisecond total latency for an end-to-end warm request would require controlled benchmarks that are not part of this thesis. What the architecture does support is that the WebAssembly execution step itself - instantiation, memory transfer, function invocation, and cleanup - can be very fast for small modules, particularly once the network and TLS components are accounted for separately.

4.5 Security

wasmCat's security model is built around explicit trust boundaries between clients, the master, and workers. The most important of these is mutual TLS. Both the master and worker servers require a verified client certificate before any request reaches an application handler. All internal control traffic is therefore encrypted, and every peer must authenticate at the transport layer.

The master additionally validates worker identity at the application level. For registration, heartbeat, drain, and completion callback endpoints, the worker identifier claimed in the JSON body must match the common name or DNS subject alternative name in the client certificate. Without this binding, a misconfigured or malicious node could submit any worker identifier it liked and interfere with another node's registry state. wasmCat rejects any request where the certificate identity and the claimed identifier do not agree.

Workers are protected in the other direction as well. The dispatcher forwards execution requests to workers over mTLS, so a worker can verify that requests originate from a certificate signed by the cluster CA rather than from arbitrary network clients. Completion callbacks from workers to the master follow the same pattern.

```
azureuser@wasmcat-mCFLJ2I:~/demo$ sudo curl -k https://localhost:7270/wasmcat/ready
curl: (56) OpenSSL SSL_read: OpenSSL/3.0.13: error:0A00045C:SSL routines::tlsv13 alert certificate required, errno 0
azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcatctl ready
status=ready role=master node_id=
```

Figure 11: mTLS enforcement: A standard curl request is rejected

Module integrity is enforced through digest verification at the worker. When a digest is provided alongside the module URL, the worker computes the SHA-256 hash of the downloaded bytes and compares it to the expected value before compilation. A mismatch causes the request to fail immediately. This matters most when modules are fetched from remote URLs or registry blobs, where content could change without the caller noticing. The digest also forms part of the cache key, which prevents a cached compiled module from being reused for a different version of the same content.

The ACR integration limits credential exposure by scoping tokens to a single repository. Rather than distributing broad Azure credentials to workers, the master mints a token scoped to the specific repository needed for the request and forwards only that token. The worker can only use it to download the blob and no other actions are allowed.

4.6 Operational Results

The operational surface of wasmCat was expanded through "wasmcatctl" and "wasmcat-ui", which remove the need for manually sending mTLS curl commands during operation.

"wasmcatctl" provides a command-line interface for health checks, readiness checks, metrics retrieval, worker listing, synchronous execution, durable job creation, durable job lookup, and worker draining. It stores configuration in a JSON file containing the master URL, CA certificate, client certificate, client key, default latitude, default longitude, and output format. This removes repeated mTLS command-line arguments and provides a reproducible operator workflow.

"wasmcat-ui" provides equivalent functionality through a browser dashboard. It includes pages for dashboard overview, workers, execution, jobs, metrics, and settings. The workers page exposes node state and drain actions. The execution page allows direct module URL or ACR-backed execution. The jobs page supports durable job lookup. The metrics page exposes master metrics in a readable form. The settings page allows the operator to view and save the same configuration format used by the CLI.

```

azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcctl execute --module jsonfmt --url http://172.16.0.4:8088/jsonfmt.wasm --payload '{"b":2,"a":1}'
request_id=req_00b46ad01836a757951e61e91c4fa3b6
node=worker-vn-01
execution_time_ms=2135.559
result={
  "a": 1,
  "b": 2
}

```

Figure 12: Synchronous execution via CLI

Figure 13: Browser dashboard

```

azureuser@wasmcat-mCFLJ2I:~/demo$ wasmcctl -o json metrics
{
  "role": "master",
  "uptime_seconds": 15536,
  "requests_total": 7522,
  "requests_by_status": {
    "200": 7516,
    "403": 5,
    "503": 1
  },
  "requests_by_path": {
    "/api/v1/execute": 9,
    "/api/v1/workers": 8,
    "/api/v1/workers/worker-vn-01/drain": 4,
    "/internal/drain": 3,
    "/internal/heartbeat": 3744,
    "/internal/jobs/complete": 6,
    "/internal/register": 3744,
    "/wasmcat/metrics": 2,
    "/wasmcat/ready": 2
  },
  "master": {
    "active_workers": 1,
    "workers_by_state": {
      "ready": 1
    },
    "oldest_heartbeat_seconds": 2,
    "dispatch_success": 5,
    "dispatch_failure": 1,
    "dispatch_reschedules": 0,
    "dispatch_reschedule_exhausted": 0,
    "request_cache_hits": 0,
    "request_in_progress_conflicts": 0,
    "request_id_conflicts": 0,
    "request_tracker": {
      "entries": 0,
      "in_flight": 0,
      "completed": 0,
      "max_entries": 4096,
      "ttl_seconds": 300,
      "evictions": 0,
      "expired": 0
    }
  }
}

```

Figure 14: Master node metrics output

Metric	Value
Uptime (s)	15 536
Total HTTP requests	7 522
status 200	7 516
status 403	5
status 503	1
Dispatch successes	5
Dispatch failures	1
Dispatch reschedules	0
Request cache hits	0
Request-ID conflicts	0

Table 8: Master runtime metrics snapshot during evaluation

4.7 Limitations

Currently, the system has implemented geo-aware scheduling, embeds a WebAssembly runtime directly in the worker binary, secures all internal communication through mutual TLS, and provides durable job tracking. However, at the moment, the most significant limitation is the single-master control plane. As the master processes important workloads, if the master process stops, no new execution requests can be accepted until it restarts. The durable SQLite store means that job records survive a process restart, and the five-second heartbeat interval means the worker registry is reconstructed quickly once the master is back. However, SQLite is local to the master host. A permanent host failure takes both the control plane and the job database with it unless external backups are in place. Moving to a high-availability design would require leader election and an external hosted job store.

The in-memory worker registry also carries a related limitation. It is efficient for live scheduling because worker state changes frequently, but it provides no persistent membership history and no stronger leasing semantics beyond the heartbeat timeout. After a master restart, membership is rebuilt from scratch as workers re-register.

The scheduling model is intentionally simple. Haversine distance gives the system a geographically-aware proxy for proximity, which satisfies the research objective of location-sensitive placement at the edge. It does not, however, capture actual network latency, which depends on routing topology, congestion, cloud peering arrangements, and current worker load. The scheduler also has no visibility into cache locality: two workers at the same distance from the user are treated identically even if one already has the requested module compiled and ready. Incorporating measured round-trip time or cache state signals into scheduling decisions would produce better placement outcomes and is a natural direction for follow-on work.

Resource isolation is limited by the current telemetry model. `gopsutil` reports host-level CPU and memory availability, which gives the scheduler a coarse signal but does not enforce per-invocation resource limits at the operating system level. WebAssembly provides memory isolation between module instances, and `wasmCat` adds application-level controls through execution timeouts, payload and output size limits, a concurrency semaphore, and module cache bounds. However, a module that consumes excessive CPU within its timeout window will compete with other invocations on the same host. Stronger governance - cgroup controls, runtime fuel metering, or admission policies that stop routing to overloaded workers - would be required for a multi-tenant production environment.

V/ CONCLUSION & PERSPECTIVE

This thesis presented the design and implementation of wasmCat, a master-worker orchestrator that runs WebAssembly modules on distributed edge workers without a container runtime. The master is the control plane. It validates requests, tracks worker state from heartbeats, schedules work by capacity and geographic distance, resolves module sources, and dispatches over mutual TLS. When a request refers to Azure Container Registry, the master provisions the module just in time: it exchanges an Azure identity token for a repository-scoped pull token and resolves the OCI manifest to a blob. The worker is the data plane. It embeds the wazero runtime, caches compiled modules under a digest-aware key, and runs each request in a fresh instance bounded by explicit resource limits. Two execution modes are supported: the custom ABI, which exchanges data through linear memory using `malloc` and `run`, and standard WASI, which uses standard input and output. A durable SQLite job store provides reliability through request idempotency, dispatch leasing, and a recovery loop. Interrupted jobs are retried, and jobs whose outcome cannot be confirmed are marked ambiguous rather than assumed complete.

The system was deployed and evaluated on Azure virtual machines in two regions, Singapore and West US, connected through VNet peering. The end-to-end path behaves as expected: a worker registers with the master, the master routes each request to the nearest eligible worker, and the module executes and returns its result. Measured execution times confirm the value of the compiled-module cache, since warm requests complete in a few milliseconds while cold requests are dominated by module download and compilation.

waswCat nonetheless remains a prototype with well-defined constraints. The control plane is a single master backed by a local SQLite database, so the loss of the master host removes both request admission and durable job history. The scheduler ranks workers by great-circle distance, which approximates network latency rather than measuring it. Resource accounting is coarse, because CPU and memory are reported per host rather than per invocation, so a single module can saturate a worker within its timeout. Finally, the deployable unit is a stateless function with no network, filesystem, or environment access, so the orchestrator can just run serverless functions, not traditional services or applications.

These constraints define the directions for future work. The most important is control-plane availability: leader election across several masters, together with a replicated or external job store, would remove the single point of failure. Scheduling could be improved by adding measured round-trip time, queue depth, historical execution time, and cache locality, so that placement reflects performance rather than position alone. Multi-tenant operation would require stronger isolation, so that per-invocation limits are enforced rather than estimated. The execution model could also be widened. An HTTP handler ABI would let modules serve web endpoints directly, and broader WASI support, including a controlled filesystem and environment policy and, in time, the component model and WASI-HTTP, would admit a larger class of modules.

In summary, wasmCat is neither a replacement for container orchestration nor a production-ready platform. It does show that a compact native Go system can implement the essential mechanics of WebAssembly-based edge serverless execution: capacity- and location-aware scheduling, just-in-time module provisioning, mutual-TLS communication, and durable request handling.

REFERENCES

- [1] Satyanarayanan, M., “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9.
- [2] Satyanarayanan, M., Bahl, P., Caceres, R., and Davies, N., “The case for vm-based cloudlets in mobile computing,” *IEEE Pervasive Computing*, vol. 8, no. 4, pp. 14–23, 2009. DOI: 10.1109/MPRV.2009.82.
- [3] Jonas, E., Schleier-Smith, J., Sreekanti, V., et al., “Cloud programming simplified: A berkeley view on serverless computing,” *arXiv preprint arXiv:1902.03383*, 2019.
- [4] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J., “Borg, omega, and kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016. DOI: 10.1145/2890784.
- [5] Sinnott, R. W., “Virtues of the haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.
- [6] Haas, A., Rossberg, A., Schuff, D. L., et al., “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 185–200. DOI: 10.1145/3062341.3062363.
- [7] W3C WebAssembly Working Group, *WebAssembly Core Specification*, <https://www.w3.org/TR/wasm-core-2/>, 2022.
- [8] Tetrate Labs, *wazero: The zero dependency WebAssembly runtime for Go developers*, <https://github.com/tetratelabs/wazero>.
- [9] National Institute of Standards and Technology, “Zero Trust Architecture,” NIST Special Publication 800-207, Tech. Rep., 2020. DOI: 10.6028/NIST.SP.800-207.
- [10] The Go Authors, *The Go Programming Language Specification*, <https://go.dev/ref/spec>.
- [11] SQLite Consortium, *SQLite Documentation*, <https://www.sqlite.org/docs.html>.
- [12] modernc.org, *SQLite driver for Go*, <https://pkg.go.dev/modernc.org/sqlite>.
- [13] Microsoft, *Azure SDK for Go, azidentity*, <https://pkg.go.dev/github.com/Azure/azure-sdk-for-go/sdk/azidentity>.
- [14] Microsoft, *Azure Container Registry documentation*, <https://learn.microsoft.com/en-us/azure/container-registry/>.
- [15] Shirou, *gopsutil v4 Go package documentation*, <https://pkg.go.dev/github.com/shirou/gopsutil/v4>.

- [16] The Go Authors, *Package net/http*, <https://pkg.go.dev/net/http>.
- [17] The Go Authors, *Package crypto/tls*, <https://pkg.go.dev/crypto/tls>.
- [18] The Go Authors, *Package sync*, <https://pkg.go.dev/sync>.
- [19] WebAssembly Community Group, *WebAssembly System Interface (WASI)*, <https://wasi.dev/>.
- [20] TinyGo Project, *TinyGo: Go compiler for small places*, <https://tinygo.org/>.
- [21] Open Container Initiative, *OCI Image Format Specification*, <https://github.com/opencontainers/image-spec>.
- [22] Rescorla, E., *The Transport Layer Security (TLS) Protocol Version 1.3*, <https://datatracker.ietf.org/doc/html/rfc8446>, 2018.
- [23] Cooper, D., Santesson, S., Farrell, S., et al., *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, <https://datatracker.ietf.org/doc/html/rfc5280>, 2008.
- [24] ORAS Project, *ORAS: OCI Registry As Storage*, <https://oras.land/>.

APPENDICES

Appendix 1: Master Configuration Reference

The master control plane is configured entirely through environment variables, read once at startup by `internal/config/config.go`. Table 9 lists each variable, its default, and its effect.

Variable	Default	Purpose
MASTER_PORT	7270	HTTPS listen port for the gateway API
CERT_DIR	./certs	Directory holding the mTLS certificate bundle
AUTO_GENERATE_CERTS	true	Generate development certificates if absent
CLEANUP_INTERVAL	15s	Interval of the stale-worker cleanup loop
WORKER_STALE_TIMEOUT	30s	Age after which a silent worker is removed
MASTER_SHUTDOWN_TIMEOUT	5s	Graceful shutdown budget
MIN_WORKER_CPU_FREE	0	Minimum free CPU (%) for scheduling eligibility
MIN_WORKER_RAM_FREE_MB	0	Minimum free RAM (MiB) for scheduling eligibility
EXECUTE_CLIENT_ALLOWLIST	(empty)	Permitted client identities for <code>/api/v1/*</code>
MAX_EXECUTION_REQUEST_BYTES	2 MiB	Maximum accepted request body size
EXECUTION_REQUEST_CACHE_TTL	5m	Idempotency cache entry lifetime
EXECUTION_REQUEST_CACHE_MAX_ENTRIES	4096	Idempotency cache capacity
JOB_STORE_PATH	./wasmcat-jobs.db	SQLite durable job database path
JOB_MAX_ATTEMPTS	3	Maximum dispatch attempts per durable job
JOB_LEASE_TTL	30s	Dispatch lease duration before a job is ambiguous
JOB_RECOVERY_INTERVAL	5s	Interval of the durable job recovery scan
JOB_RECOVERY_BATCH_SIZE	32	Jobs processed per recovery scan
MODULE_HOST_ALLOWLIST	(empty)	Permitted module source hostnames
REQUIRE_MODULE_DIGEST	false	Reject requests without a module digest

Table 9: Master environment configuration

Appendix 2: Worker Configuration Reference

Each worker node is configured as shown in Table 10. Location may be provided explicitly or auto-detected from a public geolocation provider.

Variable	Default	Purpose
WORKER_ID	worker-vn-01	Stable node identifier (must match cert filename)
WORKER_PORT	7271	HTTPS listen port for the worker API
MASTER_URL	https://localhost:7270	Master address for register/heartbeat
WORKER_ADVERTISE_ADDRESS	(host:port)	Address the master uses to reach the worker
WORKER_LATITUDE	(unset)	Explicit latitude (set with longitude)
WORKER_LONGITUDE	(unset)	Explicit longitude (set with latitude)
WORKER_AUTO_DETECT_LOCATION	true	Auto-detect coordinates when unset
WORKER_LOCATION_PROVIDER_URL	ipapi.co, ipinfo.io	Ordered geolocation providers (fall-back)
HEARTBEAT_INTERVAL	5s	Interval between heartbeat messages
EXECUTION_TIMEOUT	5s	Maximum single-invocation execution time
MODULE_FETCH_TIMEOUT	10s	Maximum module download time
MAX_MODULE_BYTES	10 MiB	Maximum module size
MAX_PAYLOAD_BYTES	1 MiB	Maximum request payload size
MAX_OUTPUT_BYTES	1 MiB	Maximum module output size
MAX_CONCURRENT_EXECS	4	Concurrent execution semaphore size
MAX_CACHED_MODULES	128	Compiled-module cache entry cap
MAX_CACHE_BYTES	256 MiB	Compiled-module cache size cap
MODULE_CACHE_TTL	30m	Compiled-module cache eviction age

Table 10: Worker environment configuration

Appendix 3: HTTP API Reference

Table 11 lists the endpoints exposed by the master gateway (port 7270) and the worker (port 7271). All endpoints require mutual TLS.

Appendix 4: Deployment and Reproduction Runbook

The following commands reproduce a two-node cluster. Certificates are issued from a single certificate authority; the CA private key never leaves the PKI directory.

```
# 1. Issue the certificate bundle from one CA
sh scripts/gen-certs.sh ca
sh scripts/gen-certs.sh master --ip <master-ip>
sh scripts/gen-certs.sh worker --id worker-vn-01 --ip <worker-ip>
```

Method	Path	Purpose
GET	/wasmcat/health	Master liveness
GET	/wasmcat/ready	Master readiness
GET	/wasmcat/metrics	Master metrics snapshot
POST	/api/v1/execute	Synchronous execution
POST	/api/v1/jobs	Create durable asynchronous job
GET	/api/v1/jobs/{request_id}	Query durable job status
GET	/api/v1/workers	List registered workers
POST	/api/v1/workers/{id}/drain	Operator-initiated worker drain
POST	/internal/register	Worker registration
POST	/internal/heartbeat	Worker heartbeat
POST	/internal/drain	Worker self-drain
POST	/internal/jobs/complete	Worker job-completion callback
POST	/invoke (worker)	Execute a module on the worker

Table 11: wasmCat HTTP API surface

```
sh scripts/gen-certs.sh operator --id wasmcat-operator

# 2. Install binaries and restart the matching service (idempotent)
sudo sh scripts/install-or-update.sh --role master --source ./dist \
  --certs-from ./wasmcat-certs/master
sudo sh scripts/install-or-update.sh --role worker --source ./dist \
  --certs-from ./wasmcat-certs/worker-worker-vn-01

# 3. Publish a module to Azure Container Registry with ORAS
oras push wasmcat.azurecr.io/modules/jsonfmt:v1 \
  jsonfmt.wasm:application/wasm

# 4. Execute against the nearest worker
wasmcatctl execute --module jsonfmt \
  --registry-url https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1 \
  --payload '{"b":2,"a":1}' --user-lat 1.29 --user-lon 103.85
```

Appendix 5: Example Request and Response

An ExecutionRequest submitted to /api/v1/execute and the ExecutionResponse returned by the master.

```
// Request
{
  "request_id": "req_demo_001",
  "module_name": "jsonfmt",
  "module_registry_url":
    "https://wasmcat.azurecr.io/v2/modules/jsonfmt/manifests/v1",
  "abi": "",
  "payload": "{\"b\":2,\"a\":1}",
```

```
"user_lat": 1.29,  
"user_lon": 103.85  
}  
  
// Response  
{  
  "request_id": "req_demo_001",  
  "result": "{\n  \"a\": 1,\n  \"b\": 2\n}",  
  "execution_time_ms": 2.932,  
  "executed_on_node_id": "worker-vn-01"  
}
```